

Grid Maze Pathfinding Comparison

Noble Carpenter
Teagan Tobias
Christian Winchester



BFS

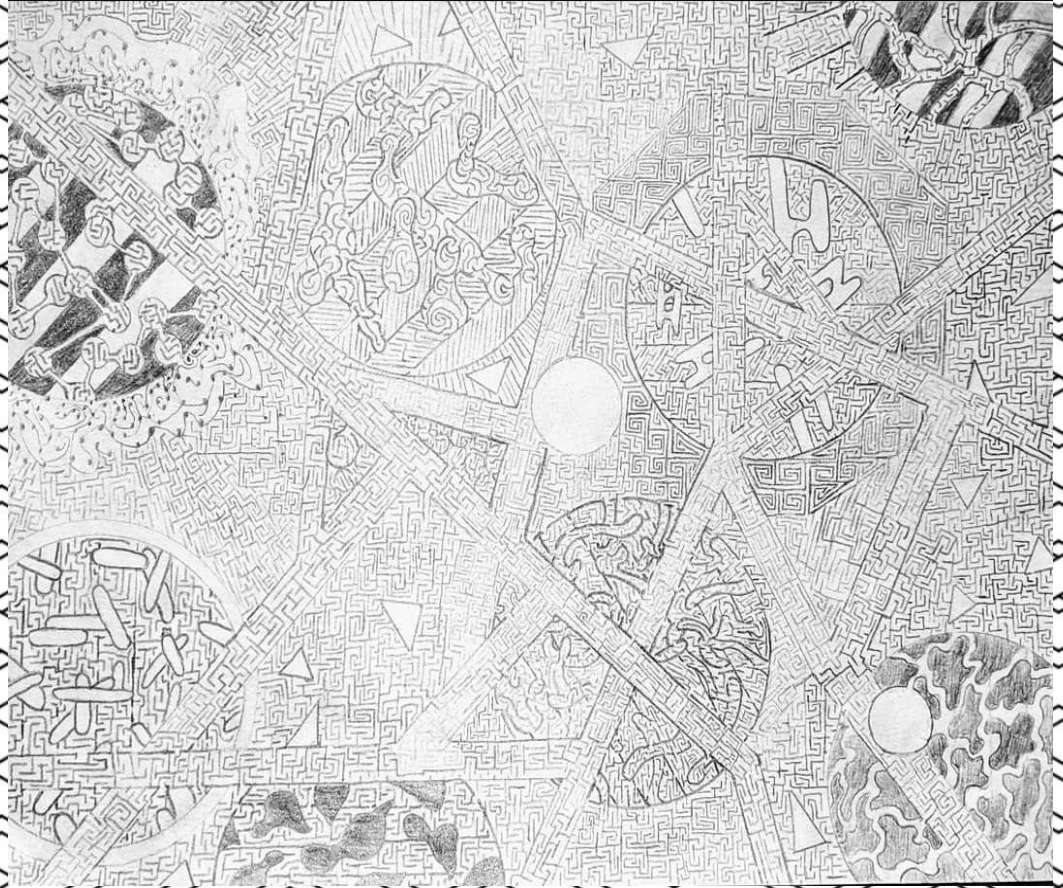
DFS

A*

ACO

The Problem

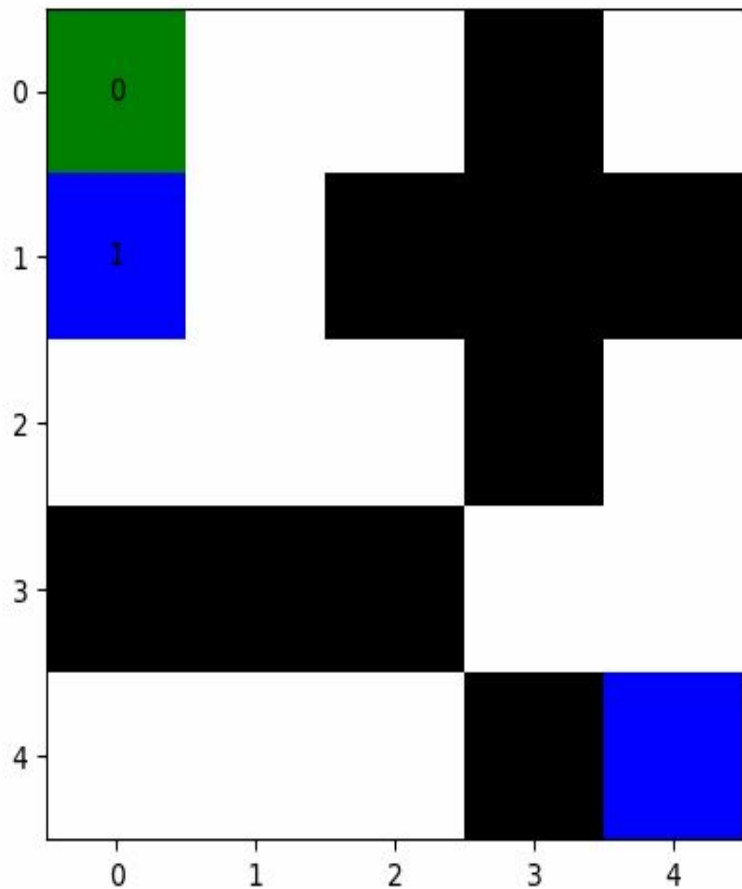
- **Objective:** Find the shortest path from start to finish in a 2D grid.
- **Constraints:** Movement restricted to four cardinal directions.
- **Cost Model:** Unweighted movement
- **Methodology:** Comparative analysis of four algorithms
- **Evaluation Metrics:** Runtime, memory usage, path optimality, and node expansion.



Breadth-First Search

Benchmark

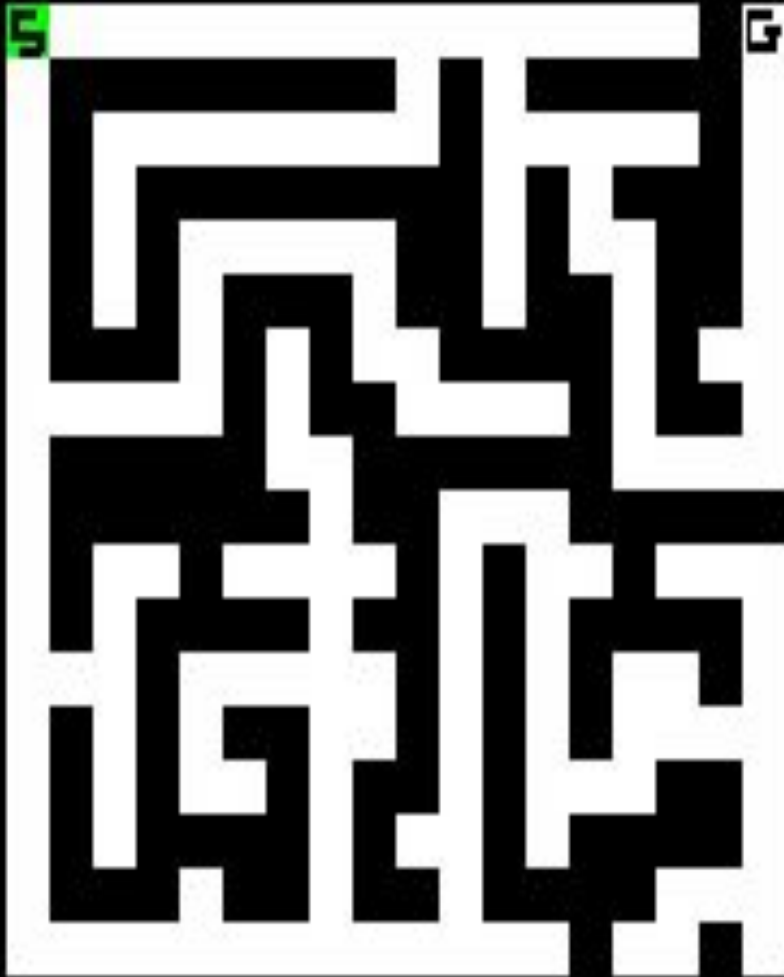
First-in-first-out queue, visiting all nodes at distance k before $k+1$



- Always finds the optimal path
 - Simple, deterministic baseline for comparison
 - Memory-intensive due to large frontier sizes
-

Time Complexity: $(V+E)$

Space Complexity: (V)



DEPTH-FIRST SEARCH

Uses last-in-first-out stack, with randomized direction order for reproducibility

- Lower memory uses than BFS
 - Often find a path quickly
 - Does not guarantee shortest path; quality degrades with size
-

Time Complexity: $(V+E)$, worst case

Space Complexity: (V) , worst case

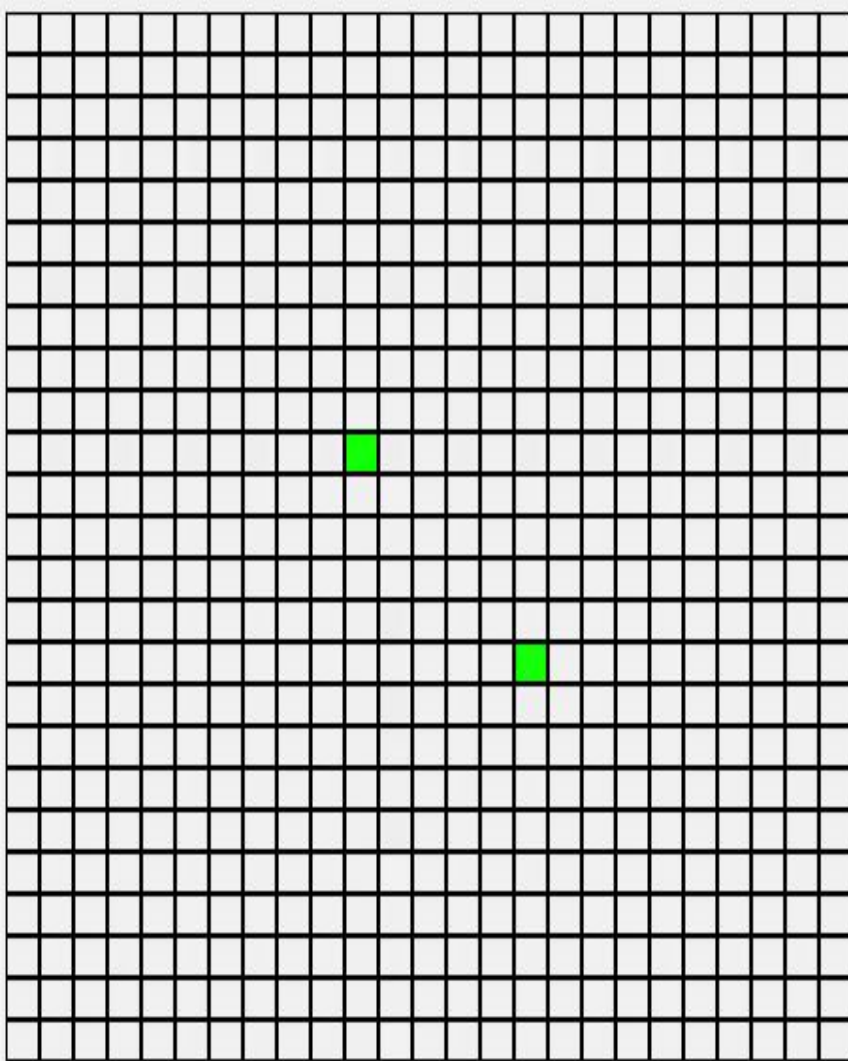
A* SEARCH

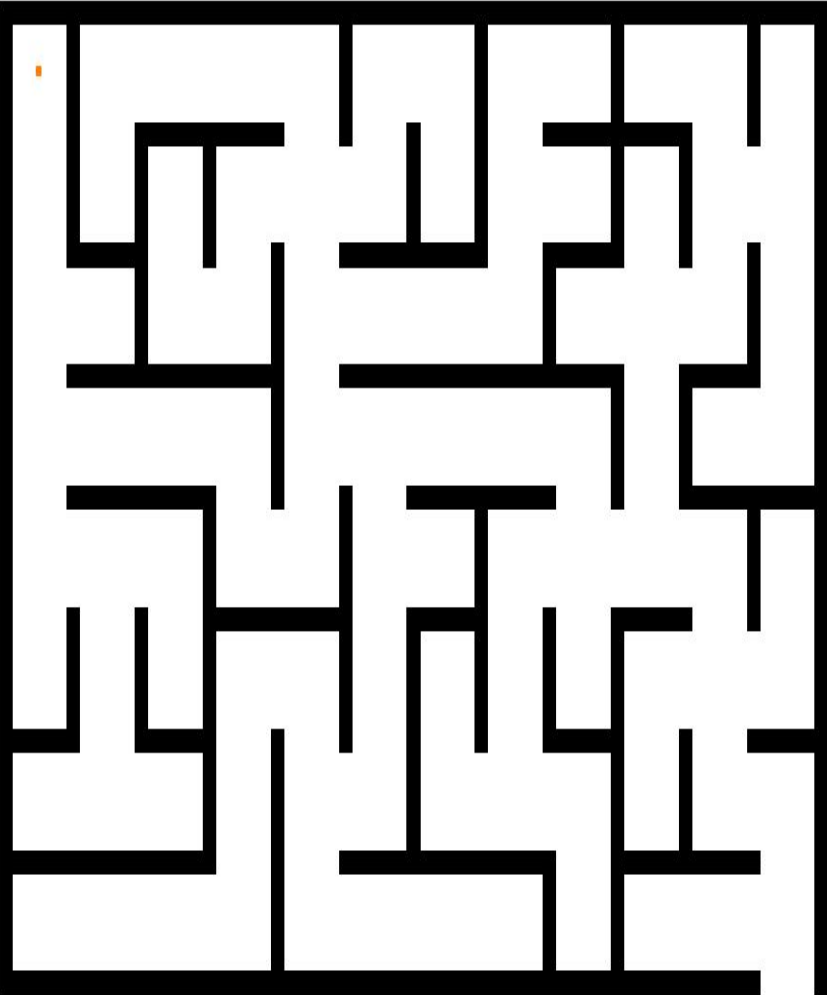
Uses a priority queue with $f(n) = g(n) + h(n)$, where $g(n)$ is path cost and $h(n)$ is Manhattan distance.

- Optimal and usually faster than BFS
 - Expands fewer nodes using heuristic guidance
 - Performance drops as obstacles increase and heuristic becomes less informative
-

Time Complexity: $(V+E)$, worst case

Space Complexity: (V)





ANT COLONY OPTIMIZATION

Nature inspired

*Paths are built probabilistically using
pheromone levels and heuristic distance*

- Bio-inspired, does not rely on explicit graph traversal
 - Stochastic and parameter-dependent
 - No guarantee of optimality
-

Time Complexity: (iterations x ants x N^2) (~constant factor ~1000 x BFS)

Space Complexity: (N^2) for pheromone matrix

Parameter	Values
Maze Sizes	10x10, 20x20, 40x40, 80x80, 160x160 (grid cells)
Obstacle Densities	10%, 20%, 30%, 40%
Trials per Configuration	20 (independent seeds per trial)
Total Experiment Runs	5 sizes x 4 densities x 4 algorithms x 20 trials = 1600 rows
Start / Goal	(0,0) $\rightarrow$ (N-1, N-1)
Seed Formula	(rows x 1000 + cols) x 100 +

Input configuration

Implementations

Programming Language:

Python 3.x

Reference Method: for

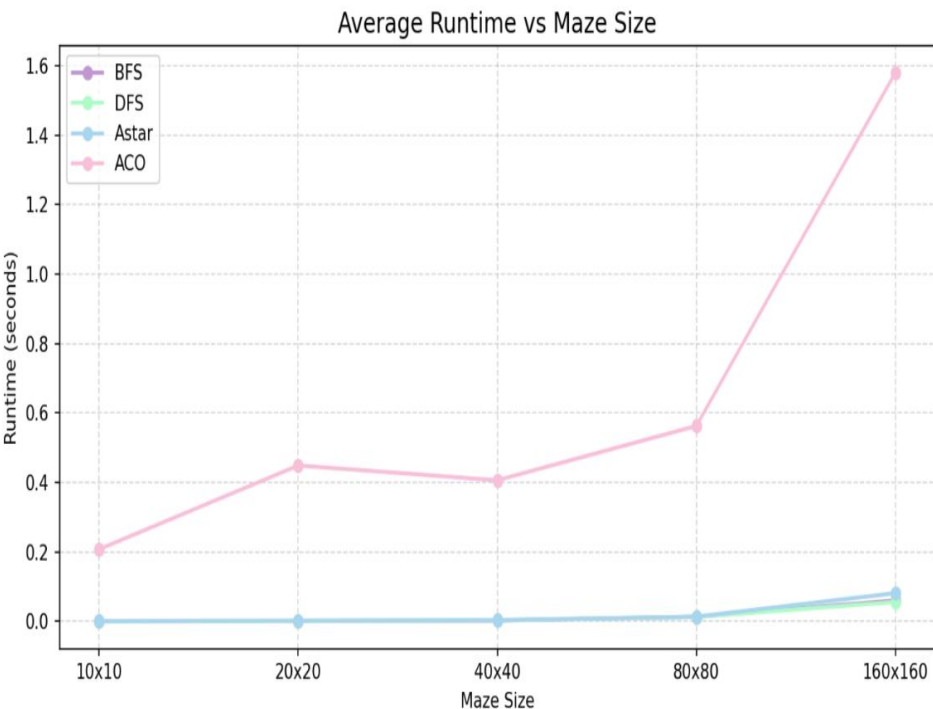
optimality, BFS, which provides the true shortest path in unweighted grids, and all other algorithms were compared against it.

Parameter	Value	Rationale
Number of Ants	20	Sufficient colony diversity without excessive runtime
Iterations	50	Allows pheromone convergence on smaller grids
Pheromone Weight	1.0	Standard equal weighting
Heuristic Weight	2.0	Stronger goal-direction pull to help ants find paths
Evaporation Rate	0.5	Moderate evaporation; tested default
Pheromone Deposit	100.0	Scales deposit relative to path length
Max Steps per ant	NxN	Prevents infinite loops on unsolvable branches

ACO Parameters

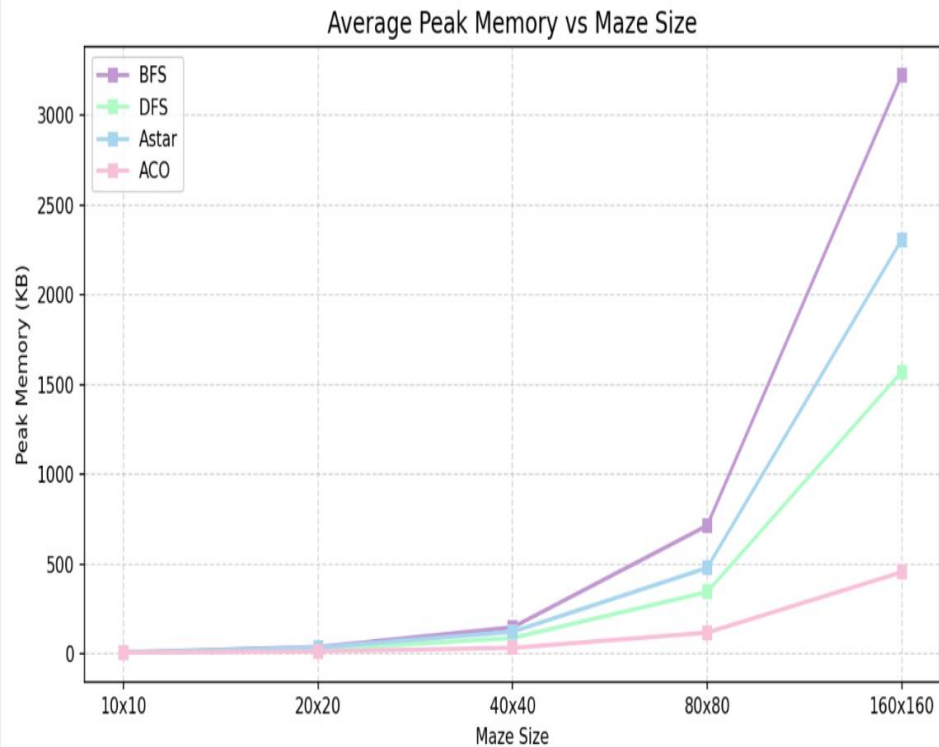
Execution Time vs Maze Size

As seen from both diagrams, the ant colony optimization runs quite slowly while everything else stays relatively equivalent



Maze Size	BFS (s)	DFS (s)	A* (s)	ACO (s)
10x10	0.000110	0.000170	0.000190	0.1512
20x20	0.000350	0.000510	0.000540	0.2990
40x40	0.001310	0.001860	0.001820	0.2574
80x80	0.011970	0.011740	0.011860	0.5526
160x160	0.053270	0.050970	0.070670	1.4457

Peak Memory Usage vs Maze Size



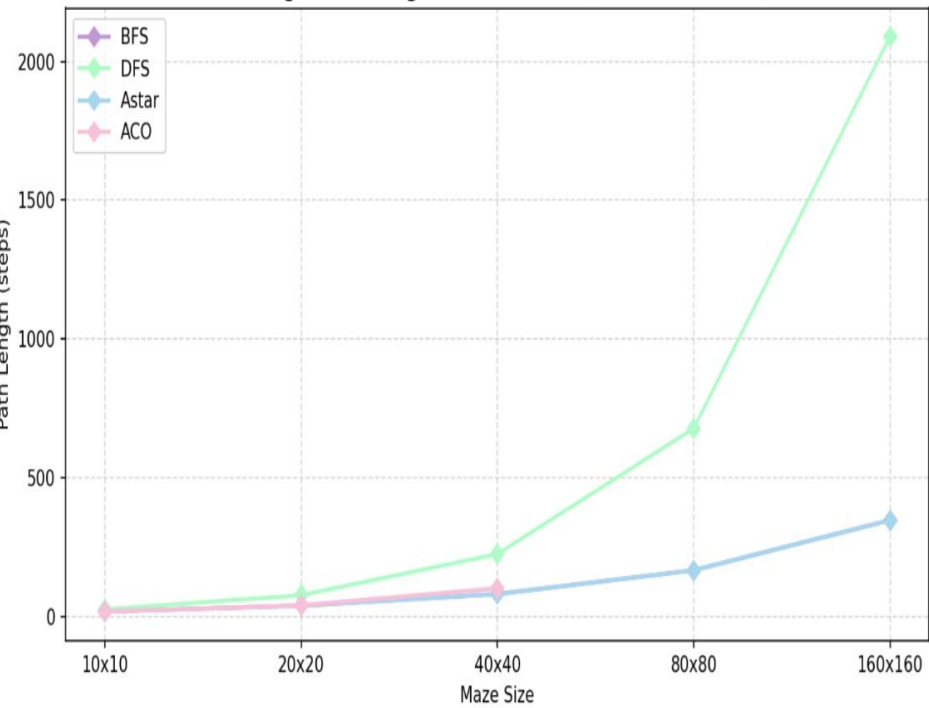
Here we can see a distinct difference between the usage of memory across all models

Maze Size	BFS (KB)	DFS (KB)	A* (KB)	ACO (KB)
10x10	9.9	7.2	9.7	7.1
20x20	38.6	23.2	36.3	15.1
40x40	148.5	89.2	123.6	33.8
80x80	714.9	345.1	479.7	118.4
160x160	3223	1571	2306	456

Path Length vs Maze Size

Interestingly, both path lengths are equal between both BFS and A*, this confirms that the Manhattan distance heuristic is admissible and consistent.

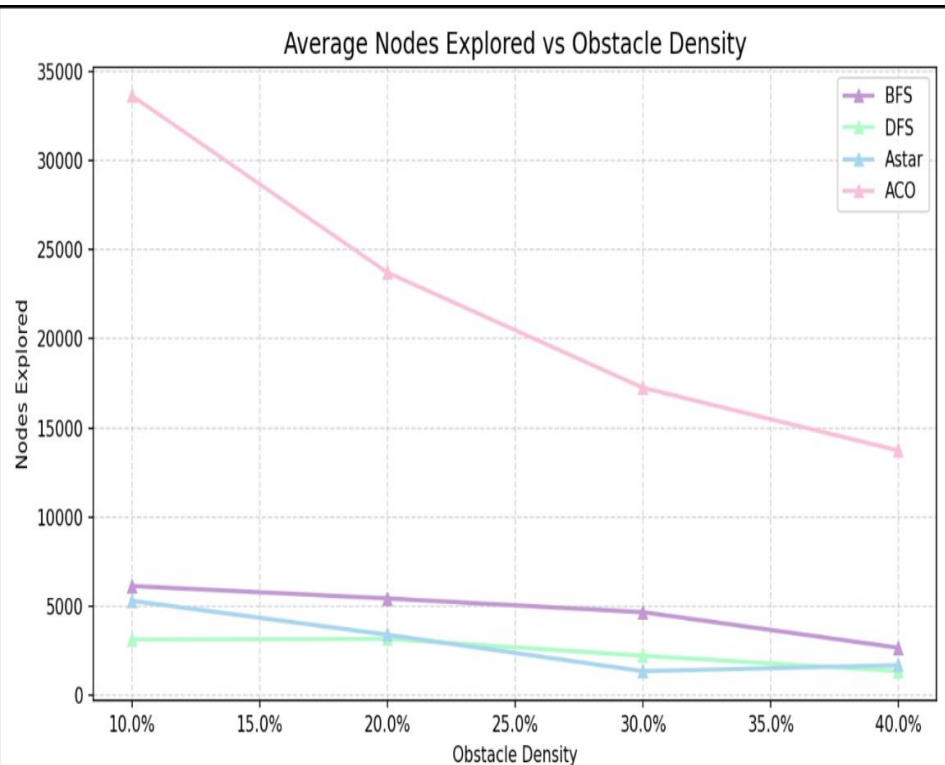
Average Path Length vs Maze Size (Successful Runs)



Maze Size	BFS (opt.)	DFS	DFS % excess	ACO	ACO % excess
10x10	18.2	25.6	+41%	18.3	+1%
20x20	39.5	77.0	+95%	39.7	+1%
40x40	81.3	225.4	+177%	103.3	+27%
80x80	166.9	677.5	+306%	N/A	N/A
160x160	347.2	2089.0	+502%	N/A	N/A

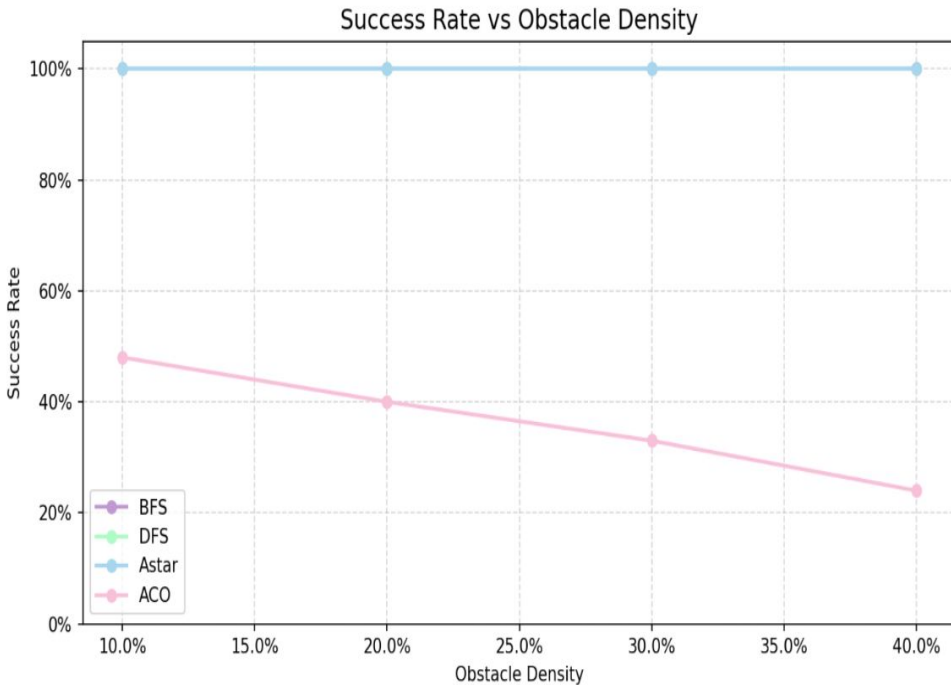
Nodes Explored vs Obstacle Density

For DFS, BFS, and A* nodes explored decrease as obstacle density increases because more walls reduce the number of reachable cells



Density	BFS	DFS	A*	ACO
10%	6133	3130	5302	32550
20%	5437	3167	3401	24014
30%	4665	2222	1356	17038
40%	2671	1364	1692	13507

Success Rate vs Obstacle Density



BFS, DFS, and A* achieve 100% success on all solvable mazes, as expected - these algorithms are complete for finite graphs.

Density	BFS	DFS	A*	ACO
10%	100%	100%	100%	49%
20%	100%	100%	100%	40%
30%	100%	100%	100%	32%
40%	100%	100%	100%	24%

Trade-off

Algorithm	Runtime	Memory	Path Quality	Success	Complexity
BFS	Fast	Highest	Optimal	100%	$\Theta(V + E)$
DFS	Fast	Medium	Very poor	100%	$\Theta(V + E)$
A*	Fastest*	Medium	Optimal	100%	$\Theta(V + E)^*$
ACO	Slowest	Lowest	Variable	23-49%	$\Theta(it \times ant \times V)$

As seen from the chart, A* dominates almost every category, making it the most efficient algorithm for maze solving

References

Chatgpt, chatgpt.com/.

Claude, claude.ai/login.

"Depth First Search or DFS for a Graph - Python." GeeksforGeeks, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/python/python-program-for-depth-first-search-or-dfs-for-a-graph/.

Dorigo, Marco, et al. Ant Colony Optimization - Cs.Princeton, 2006, www.cs.princeton.edu/courses/archive/fall24/cos597C/bib2/Ant_colony_optimization.pdf.

Editorial Professional Insights. "Ant Colony Optimization Python." Epsilon, epsilon, epsilon.newpointe.org/resource/F4D/789/read_cc3X3w_AntColonyOptimizationPython.

"Introduction to Ant Colony Optimization." GeeksforGeeks, GeeksforGeeks, 13 Feb. 2026, www.geeksforgeeks.org/machine-learning/introduction-to-ant-colony-optimization/.

Kumar, Rajesh. The A* Algorithm: A Complete Guide | Datacamp, 2024, www.datacamp.com/tutorial/a-star-algorithm.

"Maze_dataset." Maze_dataset API Documentation, understanding-search.github.io/maze-dataset/maze_dataset.html.

McKee, Amberle. Breadth-First Search in Python: A Guide with Examples | Datacamp, 2024, www.datacamp.com/tutorial/breadth-first-search-in-python.

Various Authors. "5. the Import System." Python Documentation, docs.python.org/3/reference/import.html.

Various Authors. "Random - Generate Pseudo-Random Numbers." Python Documentation, docs.python.org/3/library/random.html.

Various Authors. "TRACEMALLOC - Trace Memory Allocations." Python Documentation, docs.python.org/3/library/tracemalloc.html.